

Humanoid Simulation — Repository Structure

Unitree H1-2 humanoid sim stack — MuJoCo / Isaac physics + ROS 2 (Humble), in Docker, over CycloneDDS.

1 What it is

A physics simulator publishes the robot's state and sensors; a ROS 2 workspace does perception, planning and control. Each part runs in its own Docker container and they communicate over a shared CycloneDDS domain (default `ROS_DOMAIN_ID=1` ; `0` is reserved for the real robot). MuJoCo and Isaac are **interchangeable** physics backends — you run one.

2 Docker: how many containers, and what's in them

There are **3 runnable image profiles** plus **1 build-only base image**:

Image	Run?	Contains / does
humanoid_sim_base	no (build only)	Shared base: CUDA 12.2, Ubuntu 22.04, ROS 2 Humble, CycloneDDS 0.10.x (from source), torch cu130, pin/pink/mink, unitree_sdk2py.
humanoid_sim_mujoco	yes	FROM base + MuJoCo 3.3.1 + Robocasa. Runs the physics sim; publishes state + camera + lidar + TF.
humanoid_ros	yes	FROM base + Nav2/SLAM + vision ML. Runs the ROS 2 bringup: control, perception, planning, safety.
humanoid_sim_isaac	yes (alt)	Standalone (conda Py3.11): Isaac Sim 5.1 + IsaacLab 2.3.2; ROS via OmniGraph. Replaces mujoco.

At runtime you launch: MuJoCo mode → 2 containers (mujoco + ros) | Isaac mode → 1 container (isaac , self-contained).

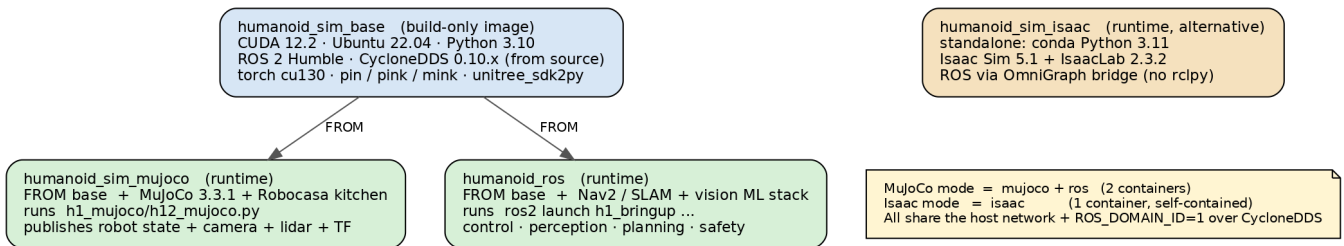


Figure 1 — Docker image inheritance and what each container holds.

3 Repository layout

Humanoid_Simulation/

- └─ docker/ Dockerfiles + compose + run scripts
 - └─ BaseDockerfile shared base image (mujoco + ros inherit)
 - └─ MujocoDockerfile RosDockerfile IsaacDockerfile
 - └─ docker-compose.yml 3 profiles: mujoco | ros | isaac
 - └─ scripts/ docker_build.sh docker_run.sh launch_{mujoco,ros,isaac}.sh
- └─ h1_mujoco/ [submodule] MuJoCo sim + DDS/ROS bridges
- └─ CL_isaaclab_sim/ [submodule] Isaac Lab tasks (alt backend)
- └─ CL_Assets/ [submodule] URDF, MuJoCo XML, meshes, scenes
- └─ container_cache/ persisted msgs_ws build artifacts
- └─ core_ws/ ROS 2 (Humble) workspace
 - └─ src/
 - └─ h1_bringup/ top-level launch + tasks (open_fridge)
 - └─ h12_lowerbody_controller/ lower-body RL: walk + FAME + switcher
 - └─ h12_ros2_controller/ [sm] arm IK (frame_task_server)
 - └─ h12_ros2_model/ [sm] robot description
 - └─ h12_safety_layer/ [sm] merge legs+arms -> rt/lowcmd, estop
 - └─ vision_pipeline/ [sm] CLIP/SAM/Gemini perception
 - └─ custom_ros_messages/ [sm] FrameTask action, Query/Beliefs srvs
 - └─ magpie_msgs/ [sm] gripper messages
 - └─ unitree_ros2/ [sm] Unitree DDS LowState/LowCmd
 - └─ livox_ros_driver2/ [sm] Livox lidar driver
 - └─ FAST_LIO/ LiDAR-inertial SLAM

[sm] = git submodule

core_ws/src package	Role
h1_bringup	Top-level launch (h1_sim_bringup.launch.py), nav launch, tasks (open_fridge.py).
h12_ros2_controller	Arm control: frame_task_server (Pink differential IK + QP) on the /frame_task action.
h12_lowerbody_controller	Lower-body RL: walking + FAME standing/squat policies + the switchable controller (\$5).
h12_safety_layer	Merges leg + arm commands → rt/lowcmd ; clipping and e-stop.
vision_pipeline	CLIP / SAM2 / SAM3 + Gemini perception; /vp_* services.
custom_ros_messages , magpie_msgs	IDL: FrameTask action, Query / UpdateBeliefs srvs, gripper msgs.
unitree_ros2 , livox_ros_driver2 , FAST_LIO	Unitree DDS msgs (LowState/LowCmd), Livox driver, LiDAR-inertial SLAM.

4 Runtime architecture (data flow)

The sim publishes robot state (`rt/lowstate`) and sensors over DDS; ROS nodes consume them and the **safety layer merges** the upper-body (arm IK) and lower-body (leg policy) commands into a single `rt/lowcmd` that drives the sim.

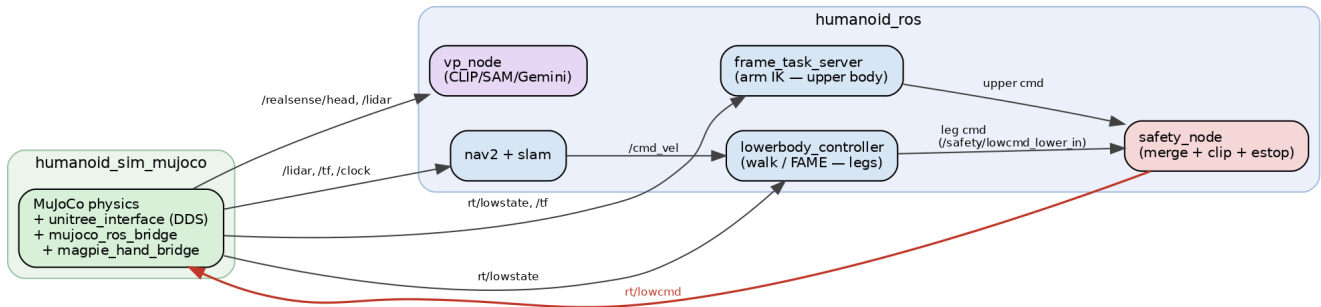


Figure 2 — Topics and nodes between the sim and the ROS stack (red = command path back to the sim).

- **State / sensors (sim → ROS):** `rt/lowstate` (joints + IMU), `/realsense/head/*`, `/lidar/points`, `/tf`, `/clock`.
- **Upper body:** `frame_task_server` solves arm IK → arm command.
- **Lower body:** `lowerbody_controller` runs an RL policy at 50 Hz → leg command on `/safety/lowcmd_lower_in`.
- **Merge & drive:** `safety_node` fuses both, clips, and publishes `rt/lowcmd` back to the sim.

5 Lower-body RL control

The legs are driven by a TorchScript RL policy. The package wraps two policies behind a service-driven state machine: the robot starts **held by the elastic band, idle**, and a policy is **started via a service**, which releases the band and engages it. Switching is gated for a clean handover.

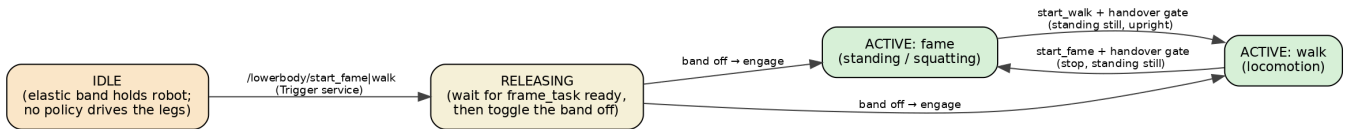


Figure 3 — Lower-body controller state machine (idle → service-activate → switch).

```

h12_lowerbody_controller/
├── policies/
│   ├── walk/    walkingPolicy.pt + walk.yaml
│   └── fame/    policy.pt + encoder_3800.pt + fame.yaml
├── h12_lowerbody_controller/
│   ├── policy.py      Policy ABC + WalkPolicy + FamePolicy
│   ├── policy_manager.py registry + idle/active + handover gate
│   ├── rma/           inference-only env-factor encoder (RMA)
│   └── scripts/
│       ├── lowerbody_controller_node.py    switchable node (used by bringup)
│       ├── walking_node.py                standalone walk node
│       └── fame_node.py                   standalone FAME node
└── reference/         standalone MuJoCo deploy scripts + standalone_fame_test.py
  
```

- **Policies:** walk (locomotion, `/cmd_vel`) and fame (RMA standing/squat; height via `/lowerbody/squat_cmd`).
- **Start:** ros2 service call `/lowerbody/start_fame std_srvs/srv/Trigger` (or `start_walk`).
- **Status / switch:** `/lowerbody/active_policy` (latched) and the `/lowerbody/set_policy` topic.
- **Gate:** a switch commits only when the robot is standing still and upright; the incoming policy is reset for a clean warm-up.

6 Example task — open the fridge

`h1_bringup/scripts/open_fridge.py` ties the whole stack together:



Figure 4 — open_fridge pipeline: navigation → vision → manipulation.